МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ
федеральное государственное бюджетное образовательное учреждение высшего образования
«Российский экономический университет имени Г.В. Плеханова»

Пермский институт (филиал)

Отчет

по производственной практике

ПП.02.01 Технология разработки программного обеспечения
(индекс по РУП и наименование производственной практики)

Профессионального модуля ПМ.02 Осуществление интеграции программных модулей

Специальность 09.02.07 «Информационные системы и программирование»

Обучающийся _____ _____Палкин Иван Андрееви
(подпись) (фамилия, имя, отчество)

Группы _Ипо-41

Руководитель практической подготовки от профильной организации

_____
(должность)

_____ _____В.О Глот_____
(подпись) (фамилия, имя, отчество)

МП «16» марта 2026 года

Руководитель практической подготовки от института

_____ _____Д.Б Берестов_____
(подпись) (фамилия, имя, отчество)

«16» марта 2026 года

Пермь, 2026 год

Оглавление

Введение 3

Глава 1 Интерфейс 4

1.1 MainWindow.xaml 4

1.2 SalesHistoryWindow.xaml 8

Глава 2 Backend 12

2.1 Mainwindow.xaml.cs 12

2.2 Palkindbcontext.cs 20

Модульные Тесты 24

DiscountServiceTests.cs 24

PatherServiceTests.cs 25

Диаграмма классов 27

Схема базы данных 28

ER-диаграмма 29

Реализация ключевых модулей 30

Тестирование программы 31

Добовления партнера 31

Проверка продаж и скидки 32

Заключение 33

Введение

Актуальность темы

В современных условиях ведения бизнеса автоматизация процессов управления партнерской сетью и расчета систем лояльности является критически важной задачей. Ручной расчет скидок и ведение учета продаж подвержены человеческим ошибкам, что может привести к финансовым потерям и снижению доверия со стороны контрагентов. Эффективная система должна не только точно рассчитывать вознаграждения в зависимости от объемов продаж, но и обеспечивать целостность данных, а также возможность быстрого масштабирования.

Цель работы

Целью данного проекта является разработка библиотеки PalkinLib на платформе .NET 8 для автоматизации управления партнерами и расчета скидок.

Задачи

Для достижения поставленной цели были решены следующие задачи:

1. **Проектирование базы данных:** Разработана схема хранения данных, включающая таблицы партнеров, типов партнеров, продаж и истории скидок.
2. **Реализация бизнес-логики:** Созданы сервисы DiscountService (расчет процентов и сумм скидок) и PartnerService (управление CRUD-операциями партнеров).
3. **Архитектурное проектирование:** Применен паттерн «Репозиторий» для абстракции доступа к данным, что позволило изолировать бизнес-логику от источника данных.
4. **Тестирование:** Написан набор юнит-тестов с использованием фреймворка xUnit и библиотеки Moq для эмуляции зависимостей, что обеспечило проверку корректности работы методов без подключения реальной базы данных.

Инструменты и технологии

В ходе разработки использовался следующий стек технологий:

- **Язык программирования:** C# (.NET 8.0).
- **Фреймворк тестирования:** xUnit.
- **Библиотека мокирования:** Moq.
- **Работа с данными:** Entity Framework Core (подразумевается) или прямой доступ через репозитории.
- **Среда разработки:** Visual Studio 2022.

Практическая значимость

Результатом работы является готовый к использованию программный модуль, который гарантирует точность расчетов скидок согласно заданным порогам (0%, 5%, 10%, 15%) и обеспечивает надежное управление данными партнеров. Покрытие тестами ключевых сценариев (добавление, обновление,

удаление, расчет) позволяет минимизировать количество ошибок при дальнейшей эксплуатации и доработке системы.

Глава 1 Интерфейс

1.1 MainWindow.xaml

```
<Window x:Class="palkinprog.MainWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:palkinprog"
        mc:Ignorable="d"
        Title="Мастер Пол - Управление партнерами"
        Height="600" Width="1000"
        WindowStartupLocation="CenterScreen">

    <Window.Resources>
        <!-- Стили для кнопок -->
        <Style x:Key="PrimaryButtonStyle" TargetType="Button">
            <Setter Property="Background" Value="#0078D4"/>
            <Setter Property="Foreground" Value="White"/>
            <Setter Property="Padding" Value="16,8"/>
            <Setter Property="Margin" Value="4"/>
            <Setter Property="BorderThickness" Value="0"/>
            <Setter Property="Cursor" Value="Hand"/>
            <Style.Triggers>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter Property="Background" Value="#106EBE"/>
                </Trigger>
            </Style.Triggers>
        </Style>

        <!-- Стили для заголовков -->
        <Style x:Key="HeaderTextStyle" TargetType="TextBlock">
            <Setter Property="FontSize" Value="16"/>
            <Setter Property="FontWeight" Value="SemiBold"/>
            <Setter Property="Margin" Value="0,0,0,8"/>
        </Style>
    </Window.Resources>

    <Window.InputBindings>
        <KeyBinding Key="F5" Command="{Binding RefreshCommand}"/>
    </Window.InputBindings>

    <Grid Margin="16">
        <Grid.RowDefinitions>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="Auto"/>
            <RowDefinition Height="*"/>
            <RowDefinition Height="Auto"/>
        </Grid.RowDefinitions>
```

```xml
        <!-- Панель инструментов -->
        <StackPanel Grid.Row="1" Orientation="Horizontal" Margin="0,0,0,16">
            <Button Content="+ Добавить партнера" Style="{StaticResource PrimaryButtonStyle}"
                    Click="AddPartnerClick"/>
            <Button Content="🔄 Обновить" Style="{StaticResource PrimaryButtonStyle}"
                    Click="RefreshClick"/>
        </StackPanel>

        <!-- Таблица партнеров -->
        <Border Grid.Row="2" BorderBrush="#E0E0E0" BorderThickness="1" CornerRadius="4">
            <DataGrid x:Name="PartnersDataGrid"
                      AutoGenerateColumns="False"
                      CanUserAddRows="False"
                      CanUserDeleteRows="False"
                      IsReadOnly="True"
                      SelectionMode="Single"
                      GridLinesVisibility="Horizontal"
                      HeadersVisibility="Column"
                      Background="White"
                      AlternatingRowBackground="#F9F9F9"
                      RowHeight="40"
                      MouseDoubleClick="PartnersDataGridMouseDoubleClick">
                <DataGrid.Columns>
                    <DataGridTextColumn Header="ID" Binding="{Binding Id}" Width="50"/>
                    <DataGridTextColumn Header="Наименование" Binding="{Binding Name}" Width="200"/>
                    <DataGridTextColumn Header="Тип партнера" Binding="{Binding PartnerTypeName}" Width="150"/>
                    <DataGridTextColumn Header="Рейтинг" Binding="{Binding Rating}" Width="70"/>
                    <DataGridTextColumn Header="Телефон" Binding="{Binding Phone}" Width="120"/>
                    <DataGridTextColumn Header="Email" Binding="{Binding Email}" Width="150"/>
                    <DataGridTextColumn Header="Общие продажи (₽)" Binding="{Binding TotalSalesAmount, StringFormat={}{0:N2}}" Width="120"/>
                    <DataGridTextColumn Header="Скидка" Binding="{Binding DiscountDescription}" Width="130"/>
                </DataGrid.Columns>
                <DataGrid.ContextMenu>
                    <ContextMenu>
                        <MenuItem Header="Редактировать" Click="EditPartnerClick"/>
                        <MenuItem Header="Продажи" Click="ShowSalesHistoryClick"/>
                        <Separator/>
                        <MenuItem Header="Удалить" Click="DeletePartnerClick"/>
                    </ContextMenu>
                </DataGrid.ContextMenu>
            </DataGrid>
        </Border>

        <!-- Статус бар -->
        <Border Grid.Row="3" Background="#F3F3F3" Padding="12,8" CornerRadius="4" Margin="0,16,0,0">
            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*"/>
                    <ColumnDefinition Width="Auto"/>
                </Grid.ColumnDefinitions>
                <TextBlock x:Name="StatusText" Grid.Column="0" Text="Готово" Foreground="#666"/>
                <TextBlock Grid.Column="1" Text="F5 - Обновить | Двойной клик - Редактировать" Foreground="#999" FontSize="12"/>
            </Grid>
        </Border>
    </Grid>
```

Это код главного экрана программы **«Мастер Пол — Управление партнерами»**.

Если представить программу как обычный рабочий стол, то этот код описывает, что именно на нём находится и как это выглядит:

1. **Шапка окна** — название программы и краткое описание, что это за раздел.
2. **Две кнопки сверху** — «Добавить партнера» и «Обновить», чтобы быстро начать работу или перезагрузить данные.
3. **Большая таблица по центру** — здесь выводится список всех партнеров. В ней видно их имя, тип, контакты, сумму продаж и текущую скидку.
4. **Управление через мышь** — по строке в таблице можно кликнуть правой кнопкой, чтобы появилось меню: «Редактировать», «Посмотреть продажи» или «Удалить». Также работает двойной клик для быстрого редактирования.
5. **Строка состояния внизу** — показывает текущий статус («Готово») и подсказки по горячим клавишам (например, F5 для обновления).

Итог: Этот файл отвечает за **внешний вид** (кнопки, таблица, цвета, отступы). Вся логика — например, как именно сохранить партнера или рассчитать скидку — находится в отдельном файле с кодом (.xaml.cs), который подключается к этому окну.

Технические детали

Теперь подробнее о том, как это реализовано технически:

Основной контейнер

xml

1

Вся разметка построена на панели Grid, разделённой на 4 строки:

1. Заголовок (авто-высота).
2. Панель кнопок (авто-высота).
3. Таблица (занимает всё свободное место *).
4. Статус-бар (авто-высота).

Стили (в <Window.Resources>)

Чтобы не дублировать настройки, созданы переиспользуемые стили:

- PrimaryButtonStyle — задаёт синий фон, белый текст и эффект при наведении для кнопок.
- HeaderTextStyle — единый шрифт и отступы для заголовков.

Горячие клавиши

xml

1

Нажатие **F5** привязано к команде обновления. Это стандартное поведение, привычное пользователям.

Таблица (DataGrid)

Самый сложный элемент окна. Ключевые настройки:

Настройка	Зачем нужна
AutoGenerateColumns="False"	Колонки заданы вручную, чтобы контролировать порядок и формат
IsReadOnly="True"	Запрет на редактирование прямо в ячейках (только через форму)
AlternatingRowBackground	Чередование цвета строк для удобства чтения
MouseDoubleClick	Обработка двойного клика для быстрого редактирования

Визуальные элементы привязаны к методам в файле MainWindow.xaml.cs через событие Click:

Элемент	Событие	Метод в коде
Кнопка «Добавить»	Click	AddPartnerClick
Кнопка «Обновить»	Click	RefreshClick
Меню «Редактировать»	Click	EditPartnerClick
Меню «Удалить»	Click	DeletePartnerClick
Двойной клик по строке	MouseDoubleClick	PartnersDataGridMouseDoubleClick

И как итог мы получаем

Мастер Пол - Управление партнерами

Управление партнерами

Подсистема работы с партнерами компании «Мастер пол»

+ Добавить партнера | Обновить

ID	Наименование	Тип партнера	Рейтинг	Телефон	Email	Общие продажи (₽)	Скидка
3	ЗАО "МеталлСервис"	Розничный магазин	28	+7 (383) 444-67-23	info@metallservice.ru	25,000.00	5% (Серебряны
2	ООО "СтройКомплект"	Розничный магазин	42	+7 (343) 555-12-89	sales@stroykomplekt.ru	5,000.00	0% (Стандарт)

Загружено партнеров: 2

F5 - Обновить | Двойной клик - Редактировать

1.2 SalesHistoryWindow.xaml

```xml
<Window x:Class="palkinprog.Dialogs.SalesHistoryWindow"
        xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
        xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
        xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
        xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
        xmlns:local="clr-namespace:palkinprog.Dialogs"
        mc:Ignorable="d"
        Title="История продаж"
        Height="550" Width="850"
        WindowStartupLocation="CenterOwner">

    <Window.Resources>
        <Style x:Key="HeaderTextStyle" TargetType="TextBlock">
            <Setter Property="FontSize" Value="16"/>
            <Setter Property="FontWeight" Value="SemiBold"/>
            <Setter Property="Margin" Value="0,0,0,8"/>
        </Style>

        <Style x:Key="InfoCardStyle" TargetType="Border">
            <Setter Property="Background" Value="#F3F3F3"/>
            <Setter Property="Padding" Value="16"/>
            <Setter Property="CornerRadius" Value="8"/>
            <Setter Property="Margin" Value="0,0,0,16"/>
        </Style>

        <Style x:Key="PrimaryButtonStyle" TargetType="Button">
            <Setter Property="Background" Value="#0078D4"/>
            <Setter Property="Foreground" Value="White"/>
            <Setter Property="Padding" Value="12,8"/>
            <Setter Property="FontSize" Value="13"/>
            <Setter Property="BorderThickness" Value="0"/>
            <Setter Property="Cursor" Value="Hand"/>
            <Style.Triggers>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter Property="Background" Value="#106EBE"/>
                </Trigger>
            </Style.Triggers>
        </Style>

        <Style x:Key="SecondaryButtonStyle" TargetType="Button">
            <Setter Property="Background" Value="#F3F3F3"/>
            <Setter Property="Foreground" Value="#333"/>
            <Setter Property="Padding" Value="12,8"/>
            <Setter Property="FontSize" Value="13"/>
            <Setter Property="BorderThickness" Value="1"/>
            <Setter Property="BorderBrush" Value="#D1D1D1"/>
            <Setter Property="Cursor" Value="Hand"/>
            <Style.Triggers>
                <Trigger Property="IsMouseOver" Value="True">
                    <Setter Property="Background" Value="#E0E0E0"/>
                </Trigger>
            </Style.Triggers>
        </Style>

        <Style x:Key="DangerButtonStyle" TargetType="Button">
            <Setter Property="Background" Value="#D13438"/>
            <Setter Property="Foreground" Value="White"/>
            <Setter Property="Padding" Value="12,8"/>
            <Setter Property="FontSize" Value="13"/>
            <Setter Property="BorderThickness" Value="0"/>
            <Setter Property="Cursor" Value="Hand"/>
```

```
<Grid Margin="16">
    <Grid.RowDefinitions>
        <RowDefinition Height="Auto"/>
        <RowDefinition Height="*"/>
        <RowDefinition Height="Auto"/>
    </Grid.RowDefinitions>

    <!-- Заголовок и информация -->
    <StackPanel Grid.Row="0">
        <Border Style="{StaticResource InfoCardStyle}">
            <Grid>
                <Grid.ColumnDefinitions>
                    <ColumnDefinition Width="*"/>
                    <ColumnDefinition Width="Auto"/>
                </Grid.ColumnDefinitions>

                <StackPanel Grid.Column="0">
                    <TextBlock x:Name="PartnerNameText" Text="Партнер" FontSize="20" FontWeight="Bold" Foreground="#333"/>
                    <TextBlock x:Name="SalesCountText" Text="Всего продаж: 0" FontSize="14" Foreground="#666" Margin="0,4,0,0"/>
                </StackPanel>

                <StackPanel Grid.Column="1" HorizontalAlignment="Right">
                    <TextBlock Text="Текущая скидка:" FontSize="14" Foreground="#666"/>
                    <TextBlock x:Name="DiscountText" Text="0%" FontSize="24" FontWeight="Bold" Foreground="#0078D4" TextAlignment="Right"/>
                </StackPanel>
            </Grid>
        </Border>

        <!-- Сводная информация -->
        <Grid Margin="0,0,0,16">
            <Grid.ColumnDefinitions>
                <ColumnDefinition Width="*"/>
                <ColumnDefinition Width="*"/>
                <ColumnDefinition Width="*"/>
            </Grid.ColumnDefinitions>

            <Border Grid.Column="0" Background="#E8F4FD" Padding="16" CornerRadius="8">
                <StackPanel>
                    <TextBlock Text="Общая сумма" FontSize="12" Foreground="#666"/>
                    <TextBlock x:Name="TotalAmountText" Text="0 ₽" FontSize="18" FontWeight="Bold" Foreground="#0078D4" Margin="0,4,0,0"/>
                </StackPanel>
            </Border>

            <Border Grid.Column="1" Background="#E8F8E8" Padding="16" CornerRadius="8" Margin="8,0">
                <StackPanel>
                    <TextBlock Text="Сумма скидки" FontSize="12" Foreground="#666"/>
                    <TextBlock x:Name="DiscountAmountText" Text="0 ₽" FontSize="18" FontWeight="Bold" Foreground="#107C10" Margin="0,4,0,0"
                </StackPanel>
            </Border>

            <Border Grid.Column="2" Background="#FFF4CE" Padding="16" CornerRadius="8">
                <StackPanel>
                    <TextBlock Text="Следующий уровень" FontSize="12" Foreground="#666"/>
                    <TextBlock x:Name="NextLevelText" Text="-" FontSize="14" FontWeight="Bold" Foreground="#856404" Margin="0,4,0,0"/>
                </StackPanel>
            </Border>
        </Grid>
    </StackPanel>

    <!-- Таблица продаж -->
    <Border Grid.Row="1" BorderBrush="#E0E0E0" BorderThickness="1" CornerRadius="4">
```

Это код всплывающего окна **«История продаж»** программы «Мастер Пол». Оно открывается, когда вы хотите посмотреть детализацию по конкретному партнёру.

Если представить это окно как отчёт, то он состоит из трёх частей:

1. **Шапка с информацией о партнёре** (верхняя серая панель)
    - Показывает имя партнёра и общее количество его продаж.
    - Справа крупно выделена его **текущая скидка** (например, «10%»), чтобы сразу было видно статус.
2. **Три цветные карточки со сводными данными** (сразу под шапкой)
    - **Общая сумма** — сколько всего денег принёс этот партнёр.
    - **Сумма скидки** — сколько денег компания «подарила» партнёру в виде скидок.
    - **Следующий уровень** — подсказка, сколько ещё нужно продать, чтобы получить более высокий процент скидки.
3. **Таблица с детализацией** (центральная часть)
    - Список всех продаж этого партнёра построчно: какой товар, сколько штук, когда продали и на какую сумму.

По строке можно кликнуть правой кнопкой мыши → появятся варианты «Редактировать» или «Удалить» запись.

4. Кнопки управления (внизу)

Добавить — внести новую продажу вручную.

Обновить — перезагрузить данные, если что-то изменилось.

Закрыть — вернуться в главное окно.

Технические детали

Структура окна

```
<Grid Margin="16">
   <Grid.RowDefinitions>
      <RowDefinition Height="Auto"/>  <!-- Шапка + карточки -->
      <RowDefinition Height="*"/>     <!-- Таблица (растягивается) -->
      <RowDefinition Height="Auto"/>  <!-- Кнопки -->
   </Grid.RowDefinitions>
</Grid>
```

Окно разделено на три логические зоны через Grid.

Стили (в <Window.Resources>)

Задано 5 переиспользуемых стилей для единообразия интерфейса:

Стиль	Назначение	Визуальные особенности
HeaderTextStyle	Заголовки	Полужирный шрифт, отступы
InfoCardStyle	Серая карточка партнёра	Фон #F3F3F3, скругление 8px
PrimaryButtonStyle	Главная кнопка («Добавить»)	Синий фон #0078D4, белый текст
SecondaryButtonStyle	Второстепенные кнопки	Светло-серый фон, рамка
DangerButtonStyle	Кнопки удаления (есть в ресурсах)	Красный фон #D13438

Верхняя информационная панель

```
<Border Style="{StaticResource InfoCardStyle}">
```

Содержит Grid с двумя колонками:

- **Слева**: имя партнёра (PartnerNameText) и количество продаж (SalesCountText).
- **Справа**: текущая скидка крупным шрифтом (DiscountText).

Три карточки статистики

Реализованы через Grid с тремя колонками (* * * — равная ширина):

Карточка	Цвет фона	Данные
Общая сумма	#E8F4FD (голубой)	TotalAmountText
Сумма скидки	#E8F8E8 (зелёный)	DiscountAmountText
Следующий уровень	#FFF4CE (жёлтый)	NextLevelText

Таблица продаж (DataGrid)

```
<DataGrid x:Name="SalesDataGrid" ...>
```

Настройки:

- AutoGenerateColumns="False" — колонки заданы вручную.
- IsReadOnly="True" — редактирование только через форму, не в ячейке.
- AlternatingRowBackground="#F9F9F9" — «зебра» для удобства чтения.
- SelectionMode="Single" — можно выбрать только одну продажу за раз.

Контекстное меню:

```
<ContextMenu>
  <MenuItem Header="Редактировать" Click="EditSaleClick"/>
  <MenuItem Header="Удалить" Click="DeleteSaleClick"/>
</ContextMenu>
```

Правый клик по строке → действия с конкретной продажей.

Кнопки управления (нижняя панель)

```
<Grid Grid.Row="2">
  <Button Content="Добавить" Style="{StaticResource PrimaryButtonStyle}" Click="AddSaleClick"/>
  <Button Content="Обновить" Style="{StaticResource SecondaryButtonStyle}" Click="RefreshClick"/>
  <Button Content="Закрыть" Style="{StaticResource SecondaryButtonStyle}" Click="CloseClick"/>
</Grid>
```

- Кнопки выровнены по правому краю через Grid.ColumnDefinitions.
- «Добавить» выделена синим как основное действие.

Связь с кодом (SalesHistoryWindow.xaml.cs)

Элемент	Событие	Метод в коде	Назначение
`AddSaleClick`	Click	`AddSaleClick`	Открыть форму добавления продажи
`RefreshClick`	Click	`RefreshClick`	Перезагрузить список из БД
`CloseClick`	Click	`CloseClick`	Закрыть окно
`EditSaleClick`	Click	`EditSaleClick`	Редактировать выбранную продажу
`DeleteSaleClick`	Click	`DeleteSaleClick`	Удалить продажу с подтверждением

Глава 2 Backend

2.1 Mainwindow.xaml.cs

```
using System.Windows;
using System.Windows.Controls;
using System.Windows.Input;
using PalkinLib.Services;
using palkinprog.Dialogs;

namespace palkinprog;


public partial class MainWindow : Window
{
    private readonly IPartnerService _partnerService;
    private List<PartnerViewModel> _partners = new();

    public MainWindow()
    {
        InitializeComponent();
        _partnerService = ((App)Application.Current).GetService<IPartnerService>()!;
        Title = "Мастер Пол - Управление партнерами";
        Loaded += MainWindowLoaded;
    }

    private async void MainWindowLoaded(object sender, RoutedEventArgs e)
    {
        await LoadPartnersAsync();
    }


    private async Task LoadPartnersAsync()
    {
        try
        {
            StatusText.Text = "Загрузка данных...";
            _partners = await _partnerService.GetAllPartnersAsync();
            PartnersDataGrid.ItemsSource = null;
            PartnersDataGrid.ItemsSource = _partners;
            StatusText.Text = $"Загружено партнеров: {_partners.Count}";
        }
        catch (Exception ex)
        {
            StatusText.Text = "Ошибка загрузки";
            MessageBox.Show(
                $"Не удалось загрузить список партнеров.\n\nОшибка: {ex.Message}\n\nПроверьте подключение к базе данных.",
                "Ошибка",
                MessageBoxButton.OK,
                MessageBoxImage.Error);
        }
    }


    private async void AddPartnerClick(object sender, RoutedEventArgs e)
    {
        await OpenPartnerEditWindowAsync(null);
    }


    private async void EditPartnerClick(object sender, RoutedEventArgs e)
    {
        if (PartnersDataGrid.SelectedItem is PartnerViewModel selectedPartner)
        {
            await OpenPartnerEditWindowAsync(selectedPartner);
        }
```

```
private async void PartnersDataGridMouseDoubleClick(object sender, MouseButtonEventArgs e)
{
    if (PartnersDataGrid.SelectedItem is PartnerViewModel selectedPartner)
    {
        await OpenPartnerEditWindowAsync(selectedPartner);
    }
}


private async void ShowSalesHistoryClick(object sender, RoutedEventArgs e)
{
    if (PartnersDataGrid.SelectedItem is not PartnerViewModel selectedPartner)
    {
        MessageBox.Show(
            "Выберите партнера для просмотра истории продаж.",
            "Предупреждение",
            MessageBoxButton.OK,
            MessageBoxImage.Warning);
        return;
    }

    try
    {
        var sales = await _partnerService.GetPartnerSalesAsync(selectedPartner.Id);
        var totalSales = await _partnerService.GetPartnerTotalSalesAsync(selectedPartner.Id);

        var salesWindow = new SalesHistoryWindow(
            selectedPartner.Name,
            selectedPartner.Id,
            sales,
            totalSales,
            selectedPartner.DiscountPercent,
            _partnerService);

        salesWindow.Owner = this;
        salesWindow.ShowDialog();
    }
    catch (Exception ex)
    {
        MessageBox.Show(
            $"Не удалось загрузить историю продаж.\n\nОшибка: {ex.Message}",
            "Ошибка",
            MessageBoxButton.OK,
            MessageBoxImage.Error);
    }
}


private async void DeletePartnerClick(object sender, RoutedEventArgs e)
{
    if (PartnersDataGrid.SelectedItem is not PartnerViewModel selectedPartner)
    {
        MessageBox.Show(
            "Выберите партнера для удаления.",
            "Предупреждение",
            MessageBoxButton.OK,
            MessageBoxImage.Warning);
        return;
    }

    var result = MessageBox.Show(
```

```
                MessageBoxImage.Warning);
            return;
        }

        var result = MessageBox.Show(
            $"Вы действительно хотите удалить партнера \"{selectedPartner.Name}\"?\n\n" +
            $"Все связанные записи о продажах также будут удалены.\n\n" +
            $"Это действие необратимо!",
            "Подтверждение удаления",
            MessageBoxButton.YesNo,
            MessageBoxImage.Warning);

        if (result == MessageBoxResult.Yes)
        {
            try
            {
                await _partnerService.DeletePartnerAsync(selectedPartner.Id);
                StatusText.Text = $"Партнер \"{selectedPartner.Name}\" удален";
                await LoadPartnersAsync();
            }
            catch (Exception ex)
            {
                MessageBox.Show(
                    $"Не удалось удалить партнера.\n\nОшибка: {ex.Message}",
                    "Ошибка",
                    MessageBoxButton.OK,
                    MessageBoxImage.Error);
            }
        }
    }

    private async void RefreshClick(object sender, RoutedEventArgs e)
    {
        await LoadPartnersAsync();
    }

    private async Task OpenPartnerEditWindowAsync(PartnerViewModel? partner)
    {
        try
        {
            var partnerTypes = await _partnerService.GetPartnerTypesAsync();
            var editWindow = new PartnerEditWindow(partner, partnerTypes, _partnerService);
            editWindow.Owner = this;

            if (editWindow.ShowDialog() == true)
            {
                await LoadPartnersAsync();
                StatusText.Text = partner == null
                    ? "Партнер добавлен"
                    : $"Партнер \"{partner.Name}\" обновлен";
            }
        }
        catch (Exception ex)
        {
            MessageBox.Show(
                $"Не удалось открыть форму редактирования.\n\nОшибка: {ex.Message}",
                "Ошибка",
                MessageBoxButton.OK,
                MessageBoxImage.Error);
        }
```

Это «мозг» главного окна программы — файл MainWindow.xaml.cs. Если предыдущий файл (XAML) отвечал за внешний вид (кнопки, таблица), то этот код отвечает за **действия**: что происходит, когда вы нажимаете на эти кнопки. Вот что умеет этот код:

1. **Запускается и загружает данные**
- Когда окно открывается, оно автоматически обращается к базе данных через сервис _partnerService и запрашивает список всех партнёров.
- Пока данные грузятся, внизу пишет «Загрузка данных...», а потом показывает результат: «Загружено партнеров: X».
- Если что-то пошло не так (например, нет связи с базой), покажет окно с ошибкой.
2. **Добавляет и редактирует партнёров**
- При нажатии на кнопку «Добавить» или «Редактировать» (в том числе по двойному клику) открывается отдельное окно (PartnerEditWindow).
- Если вы добавили нового — окно передаёт пустую форму. Если редактируете — передаёт данные выбранного партнёра.
- После сохранения окно закрывается, а список в главном окне автоматически обновляется.
3. **Показывает историю продаж**
- Когда вы выбираете партнёра и нажимаете «Продажи», код запрашивает у сервиса две вещи: список продаж и общую сумму.
- Затем создаёт окно SalesHistoryWindow, передаёт туда все данные и открывает его.
4. **Удаляет партнёров**
- Перед удалением обязательно показывает предупреждение: «Вы уверены? Все продажи тоже удалятся!».
- Если вы нажали «Да» — вызывает удаление в сервисе и обновляет список.
5. **Обрабатывает ошибки**
- Везде, где есть обращение к базе данных, стоит блок try-catch. Если случится сбой, программа не «упадёт», а покажет понятное сообщение об ошибке.

Итог: Этот класс связывает красивые кнопки на экране с реальной бизнес-логикой из библиотеки PalkinLib. Он ничего не считает сам — он только передаёт команды сервису и показывает результат пользователю.

Технические детали

Поля и конструктор

```
private readonly IPartnerService _partnerService;
private List<PartnerViewModel> _partners = new();

public MainWindow()
{
    InitializeComponent();
    // Получаем сервис через DI из App.xaml.cs
    _partnerService = ((App)Application.Current).GetService<IPartnerService>()!;
    Title = "Мастер Пол - Управление партнерами";
    // Подписываемся на событие загрузки окна
    Loaded += MainWindowLoaded;
}
```

- **IPartnerService** — интерфейс сервиса, внедряется через Dependency Injection. Это позволяет легко подменять реализацию (например, в тестах).
- **_partners** — локальный список для хранения данных, чтобы не запрашивать их из БД при каждом обращении.
- **Loaded += MainWindowLoaded** — подписка на событие: как только окно отрисовалось, сразу грузим данные.

Загрузка данных (LoadPartnersAsync)

```
private async Task LoadPartnersAsync()
{
    try
    {
        StatusText.Text = "Загрузка данных...";
        _partners = await _partnerService.GetAllPartnersAsync();

        // Перезагрузка источника данных для DataGrid
        PartnersDataGrid.ItemsSource = null;
        PartnersDataGrid.ItemsSource = _partners;

        StatusText.Text = $"Загружено партнеров: {_partners.Count}";
    }
    catch (Exception ex)
    {
        StatusText.Text = "Ошибка загрузки";
        MessageBox.Show($"Не удалось загрузить список
партнеров.\n\nОшибка: {ex.Message}", ...);
    }
}
```

- Используется async/await, чтобы интерфейс не зависал во время запроса к БД.
- ItemsSource = null перед установкой нового значения — старый трюк для WPF, чтобы таблица корректно обновила отображение.

Добавление и редактирование

```
private async void AddPartnerClick(object sender, RoutedEventArgs e)
{
    await OpenPartnerEditWindowAsync(null); // null = новый партнёр
}

private async void EditPartnerClick(object sender, RoutedEventArgs e)
{
    if (PartnersDataGrid.SelectedItem is PartnerViewModel selectedPartner)
    {
        await OpenPartnerEditWindowAsync(selectedPartner);
    }
}
```

Оба метода вызывают один общий OpenPartnerEditWindowAsync, передавая либо null (создание), либо объект партнёра (редактирование).

Удаление с подтверждением

```
private async void DeletePartnerClick(object sender, RoutedEventArgs e)
{
    if (PartnersDataGrid.SelectedItem is not PartnerViewModel selectedPartner)
    {
        MessageBox.Show("Выберите партнера для удаления.", ...);
        return;
    }

    var result = MessageBox.Show(
        $"Вы действительно хотите удалить партнера \"{selectedPartner.Name}\"?...",
        "Подтверждение удаления",
        MessageBoxButton.YesNo,
        MessageBoxImage.Warning);

    if (result == MessageBoxResult.Yes)
    {
        try
        {
            await _partnerService.DeletePartnerAsync(selectedPartner.Id);
            StatusText.Text = $"Партнер \"{selectedPartner.Name}\" удален";
            await LoadPartnersAsync(); // Обновляем таблицу
        }
        catch (Exception ex)
        {
            MessageBox.Show($"Не удалось удалить партнера.\n\nОшибка: {ex.Message}", ...);
        }
    }
}
```

Проверка на null — защита от удаления «в пустоту».

MessageBox.Show с YesNo — стандартный диалог подтверждения.

После успешного удаления — принудительная перезагрузка списка.

История продаж

```
private async void ShowSalesHistoryClick(object sender, RoutedEventArgs e)
{
    if (PartnersDataGrid.SelectedItem is not PartnerViewModel selectedPartner)
    {
        MessageBox.Show("Выберите партнера для просмотра истории продаж.", ...);
        return;
    }
```

```
    try
    {
        // Запрашиваем данные у сервиса
        var sales = await _partnerService.GetPartnerSalesAsync(selectedPartner.Id);
        var totalSales = await _partnerService.GetPartnerTotalSalesAsync(selectedPartner.Id);

        // Создаём и настраиваем окно истории
        var salesWindow = new SalesHistoryWindow(
            selectedPartner.Name,
            selectedPartner.Id,
            sales,
            totalSales,
            selectedPartner.DiscountPercent,
            _partnerService); // Передаём сервис для возможных действий внутри окна

        salesWindow.Owner = this; // Делаем окно модальным (привязываем к главному)
        salesWindow.ShowDialog(); // Показываем блокирующим образом
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Не удалось загрузить историю продаж.\n\nОшибка: {ex.Message}", ...);
    }
}
```

- ShowDialog() — открывает окно модально (пользователь не может кликнуть на главное окно, пока не закроет это).
- Сервис передаётся в окно истории, чтобы там тоже можно было, например, добавить или удалить продажу.

Общий метод открытия формы редактирования

```
private async Task OpenPartnerEditWindowAsync(PartnerViewModel? partner)
{
    try
    {
        // Загружаем справочник типов партнёров для комбобокса
        var partnerTypes = await _partnerService.GetPartnerTypesAsync();

        var editWindow = new PartnerEditWindow(partner, partnerTypes, _partnerService);
        editWindow.Owner = this;

        // Если окно закрылось с результатом OK (DialogResult = true)
```

```
        if (editWindow.ShowDialog() == true)
        {
            await LoadPartnersAsync(); // Обновляем главный список
            StatusText.Text = partner == null
                ? "Партнер добавлен"
                : $"Партнер \"{partner.Name}\" обновлен";
        }
    }
    catch (Exception ex)
    {
        MessageBox.Show($"Не удалось открыть форму редактирования.\n\nОшибка: {ex.Message}", ...);
    }
}
```

- Возвращает Task, так как внутри есть await.
- Проверяет DialogResult: если пользователь нажал «Сохранить» (и форма вернула true), обновляем данные. Если «Отмена» — ничего не делаем.

Обработчик кнопки «Обновить»

```
private async void RefreshClick(object sender, RoutedEventArgs e)
{
    await LoadPartnersAsync();
}
```

Просто вызывает перезагрузку данных. Удобно, если данные изменились в другом окне или вручную в БД.

Структура методов

Метод	Тип	Назначение
MainWindow()	Конструктор	Инициализация, получение сервиса, подписка на события
MainWindowLoaded()	Event Handler	Автозагрузка данных при открытии окна
LoadPartnersAsync()	Private Async	Основной метод загрузки и отображения списка
AddPartnerClick()	Event Handler	Открытие формы создания
EditPartnerClick()	Event Handler	Открытие формы редактирования по кнопке
PartnersDataGridMouseDoubleClick()	Event Handler	Редактирование по двойному клику
ShowSalesHistoryClick()	Event Handler	Открытие окна истории продаж
DeletePartnerClick()	Event Handler	Удаление с подтверждением
RefreshClick()	Event Handler	Принудительное обновление списка
OpenPartnerEditWindowAsync()	Private Async	Универсальный метод открытия формы редактирования

2.2 Palkindbcontext.cs

```
using PalkinLib.Models;

namespace PalkinLib.Data;

/// <summary>
/// Контекст базы данных для подсистемы работы с партнерами
/// </summary>
public class PalkinDbContext : DbContext
{
    public DbSet<Partner> Partners => Set<Partner>();
    public DbSet<PartnerType> PartnerTypes => Set<PartnerType>();
    public DbSet<Sale> Sales => Set<Sale>();

    private readonly string _connectionString;

    public PalkinDbContext(string connectionString)
    {
        _connectionString = connectionString;
    }

    protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
    {
        optionsBuilder.UseNpgsql(_connectionString);
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        // Настройка схемы
        modelBuilder.HasDefaultSchema("app");

        // Конфигурация PartnerType
        modelBuilder.Entity<PartnerType>(entity =>
        {
            entity.ToTable("partner_types");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Id).HasColumnName("id");
            entity.Property(e => e.Name).IsRequired().HasMaxLength(100).HasColumnName("name");
            entity.HasIndex(e => e.Name).IsUnique();
        });

        // Конфигурация Partner
        modelBuilder.Entity<Partner>(entity =>
        {
            entity.ToTable("partners");
            entity.HasKey(e => e.Id);
            entity.Property(e => e.Id).HasColumnName("id");
            entity.Property(e => e.Name).IsRequired().HasMaxLength(255).HasColumnName("name");
            entity.Property(e => e.LegalAddress).HasMaxLength(500).HasColumnName("legal_address");
            entity.Property(e => e.Inn).HasMaxLength(20).HasColumnName("inn");
            entity.Property(e => e.DirectorName).HasMaxLength(255).HasColumnName("director_name");
            entity.Property(e => e.Phone).HasMaxLength(50).HasColumnName("phone");
            entity.Property(e => e.Email).HasMaxLength(100).HasColumnName("email");
            entity.Property(e => e.Rating).HasColumnName("rating");
            entity.Property(e => e.CreatedAt).HasColumnType("timestamp without time zone").HasColumnName("created_at");
            entity.Property(e => e.UpdatedAt).HasColumnType("timestamp without time zone").HasColumnName("updated_at");
            entity.Property(e => e.PartnerTypeId).HasColumnName("partner_type_id");
```

Это файл настройки подключения к базе данных — «переводчик» между вашим C#-кодом и реальной базой PostgreSQL.

Если представить базу данных как склад с таблицами, то этот класс (PalkinDbContext) делает три важные вещи:

1. **Подключается к базе**
    - Принимает строку подключения (адрес сервера, логин, пароль, имя базы).
    - Говорит программе: «Мы работаем с PostgreSQL» (через UseNpgsql).
2. **Показывает, какие таблицы есть в базе**
    - Partners → таблица partners (партнёры)
    - PartnerTypes → таблица partner_types (типы партнёров: розница, опт и т.д.)

- Sales → таблица sales (история продаж)
- Вы обращаетесь к ним в коде как к обычным коллекциям: _context.Partners.ToList().

3. **Настраивает правила для каждой таблицы**
   - Какое поле за что отвечает (например, Name → колонка name).
   - Какие поля обязательные, а какие нет.
   - Какая максимальная длина у текста (чтобы не влезало лишнее).
   - Как таблицы связаны между собой (например, у каждой продажи есть партнёр).
   - Какие данные проверяет сама база (например, «рейтинг не может быть отрицательным»).

Технические детали

Класс и свойства

```
public class PalkinDbContext : DbContext
{
    public DbSet<Partner> Partners => Set<Partner>();
    public DbSet<PartnerType> PartnerTypes => Set<PartnerType>();
    public DbSet<Sale> Sales => Set<Sale>();

    private readonly string _connectionString;

    public PalkinDbContext(string connectionString)
    {
        _connectionString = connectionString;
    }
```

- Наследуется от DbContext — базового класса Entity Framework Core.
- DbSet<T> — представляет таблицу в базе. Позволяет делать запросы: _context.Partners.Where(p => p.Rating > 5).
- Конструктор принимает строку подключения — так её можно менять без перекомпиляции (например, для тестов и продакшена).

Настройка подключения (OnConfiguring)

```
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    optionsBuilder.UseNpgsql(_connectionString);
}
```

- Указывает провайдер базы данных: Npgsql для PostgreSQL.
- Если в будущем перейдёте на SQL Server — поменяете только эту строку на UseSqlServer(...).

Настройка схемы и таблиц (OnModelCreating)

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.HasDefaultSchema("app");
}
```

- HasDefaultSchema("app") — все таблицы создаются в схеме app (удобно для изоляции в одной базе нескольких приложений).

Таблица partner_types (Типы партнёров)

```
modelBuilder.Entity<PartnerType>(entity =>
{
    entity.ToTable("partner_types");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Id).HasColumnName("id");
    entity.Property(e => e.Name).IsRequired().HasMaxLength(100).HasColumnName("name");
    entity.HasIndex(e => e.Name).IsUnique();
});
```

Настройка	Что делает
ToTable("partner_types")	Имя таблицы в БД
HasKey(e => e.Id)	Первичный ключ — поле id
IsRequired()	Поле name не может быть пустым
HasMaxLength(100)	Максимум 100 символов в имени
HasIndex(...).IsUnique()	Запрещает дубликаты названий типов

Таблица partners (Партнёры)

```
modelBuilder.Entity<Partner>(entity =>
{
    entity.ToTable("partners");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Id).HasColumnName("id");
    entity.Property(e => e.Name).IsRequired().HasMaxLength(255).HasColumnName("name");
    entity.Property(e => e.LegalAddress).HasMaxLength(500).HasColumnName("legal_address");
    entity.Property(e => e.Inn).HasMaxLength(20).HasColumnName("inn");
    entity.Property(e => e.DirectorName).HasMaxLength(255).HasColumnName("director_name");
    entity.Property(e => e.Phone).HasMaxLength(50).HasColumnName("phone");
    entity.Property(e => e.Email).HasMaxLength(100).HasColumnName("email");
    entity.Property(e => e.Rating).HasColumnName("rating");
    entity.Property(e => e.CreatedAt).HasColumnType("timestamp without time zone").HasColumnName("created_at");
    entity.Property(e => e.UpdatedAt).HasColumnType("timestamp without time zone").HasColumnName("updated_at");
    entity.Property(e => e.PartnerTypeId).HasColumnName("partner_type_id");

    entity.Property(e => e.CreatedAt).ValueGeneratedOnAdd();
    entity.Property(e => e.UpdatedAt).ValueGeneratedOnAddOrUpdate();

    entity.HasOne(e => e.PartnerType)
        .WithMany(e => e.Partners)
        .HasForeignKey(e => e.PartnerTypeId)
```

```
        .OnDelete(DeleteBehavior.Restrict);

    entity.Ignore(e => e.TotalSalesAmount);
    entity.HasCheckConstraint("CK_Partners_Rating", "\"rating\" >= 0");
});
```

Поле	***Ограничение***	***Назначение***
Name	*Обязательное, до 255 симв.*	*Название партнёра*
INN	*До 20 симв.*	*ИНН организации*
Email	*До 100 симв.*	*Контактный email*
Rating	*Проверка >= 0*	*Рейтинг лояльности*
CreatedAt	*Заполняется при создании*	*Дата добавления*
UpdatedAt	*Обновляется при изменении*	*Дата последнего изменения*

Связь с типом партнёра:

```
entity.HasOne(e => e.PartnerType)
    .WithMany(e => e.Partners)
    .HasForeignKey(e => e.PartnerTypeId)
    .OnDelete(DeleteBehavior.Restrict);
```

- Один тип партнёра → много партнёров.
- Restrict — нельзя удалить тип, если есть хотя бы один партнёр этого типа (защита от случайного удаления).

Проверка на уровне БД:

```
entity.HasCheckConstraint("CK_Partners_Rating", "\"rating\" >= 0");
```

- Даже если код пропустит отрицательный рейтинг, база данных отклонит такую запись.

Таблица sales (Продажи)

```
modelBuilder.Entity<Sale>(entity =>
{
    entity.ToTable("sales");
    entity.HasKey(e => e.Id);
    entity.Property(e => e.Id).HasColumnName("id");
    entity.Property(e => e.PartnerId).HasColumnName("partner_id");
    entity.Property(e => e.ProductName).IsRequired().HasMaxLength(255).HasColumnName("product_name");
    entity.Property(e => e.Quantity).HasColumnName("quantity");
    entity.Property(e => e.SaleDate).HasColumnType("date").HasColumnName("sale_date");
    entity.Property(e => e.Amount).HasColumnName("amount");

    entity.HasOne(e => e.Partner)
        .WithMany(e => e.Sales)
        .HasForeignKey(e => e.PartnerId)
        .OnDelete(DeleteBehavior.Cascade);
```

Модульные Тесты

DiscountServiceTests.cs

```
using PalkinLib.Services;

namespace PalkinLibTests;

/// <summary>
/// Тесты для сервиса расчета скидок
/// </summary>
public class DiscountServiceTests
{
    [Theory]
    [InlineData(0, 0)]
    [InlineData(5000, 0)]
    [InlineData(9999, 0)]
    public void CalculateDiscountPercent_ReturnsZero_WhenSalesBelow10000(decimal sales, int expectedDiscount)
    {
        // Act
        var result = DiscountService.CalculateDiscountPercent(sales);

        // Assert
        Assert.Equal(expectedDiscount, result);
    }

    [Theory]
    [InlineData(10000, 5)]
    [InlineData(25000, 5)]
    [InlineData(49999, 5)]
    public void CalculateDiscountPercent_ReturnsFive_WhenSalesBetween10000And50000(decimal sales, int expectedDiscount)
    {
        // Act
        var result = DiscountService.CalculateDiscountPercent(sales);

        // Assert
        Assert.Equal(expectedDiscount, result);
    }

    [Theory]
    [InlineData(50000, 10)]
    [InlineData(150000, 10)]
    [InlineData(299999, 10)]
    public void CalculateDiscountPercent_ReturnsTen_WhenSalesBetween50000And300000(decimal sales, int expectedDiscount)
    {
        // Act
        var result = DiscountService.CalculateDiscountPercent(sales);

        // Assert
        Assert.Equal(expectedDiscount, result);
    }

    [Theory]
    [InlineData(300000, 15)]
    [InlineData(500000, 15)]
    [InlineData(1000000, 15)]
    public void CalculateDiscountPercent_ReturnsFifteen_WhenSalesAbove300000(decimal sales, int expectedDiscount)
    {
        // Act
        var result = DiscountService.CalculateDiscountPercent(sales);

        // Assert
        Assert.Equal(expectedDiscount, result);
    }

    [Fact]
    public void CalculateDiscountAmount_CalculatesCorrectly()
    {
        // Arrange
        var sales = 100000m; // 10% discount = 10000

        // Act
```

Это **статический сервис** для расчёта скидок партнёрам. Никаких зависимостей, базы данных или моков — только чистая логика.

Как считается скидка

Сумма продаж	Скидка	Статус
< 10 000 ₽	0%	Стандарт
10 000 – 49 999 ₽	5%	Серебряный партнёр
50 000 – 299 999 ₽	10%	Золотой партнёр
≥ 300 000 ₽	15%	Премиум партнёр

Три метода

Метод	Что делает	Пример
CalculateDiscountPercent(decimal)	Возвращает процент скидки	100000 → 10
CalculateDiscountAmount(decimal)	Считает сумму скидки в рублях	100000 → 10000
GetDiscountDescription(decimal)	Возврат красивого текста	50000 → "10% (Золотой партнер)"

PatherServiceTests.cs

```
using Moq;
using PalkinLib.Models;
using PalkinLib.Repositories;
using PalkinLib.Services;

namespace PalkinLibTests;

/// <summary>
/// Тесты для сервиса управления партнерами
/// </summary>
public class PartnerServiceTests
{
    private readonly Mock<IPartnerRepository> _partnerRepositoryMock;
    private readonly Mock<IPartnerTypeRepository> _partnerTypeRepositoryMock;
    private readonly Mock<ISaleRepository> _saleRepositoryMock;
    private readonly PartnerService _partnerService;

    public PartnerServiceTests()
    {
        _partnerRepositoryMock = new Mock<IPartnerRepository>();
        _partnerTypeRepositoryMock = new Mock<IPartnerTypeRepository>();
        _saleRepositoryMock = new Mock<ISaleRepository>();

        _partnerService = new PartnerService(
            _partnerRepositoryMock.Object,
            _partnerTypeRepositoryMock.Object,
            _saleRepositoryMock.Object);
    }

    [Fact]
    public async Task GetAllPartnersAsync_ReturnsViewModelsWithTotalSales()
    {
        // Arrange
        var partnerType = new PartnerType { Id = 1, Name = "Розничный магазин" };
        var partners = new List<Partner>
        {
            new() { Id = 1, Name = "Partner 1", PartnerTypeId = 1, PartnerType = partnerType, Rating = 0 },
            new() { Id = 2, Name = "Partner 2", PartnerTypeId = 1, PartnerType = partnerType, Rating = 5 }
        };

        _partnerRepositoryMock.Setup(r => r.GetAllAsync()).ReturnsAsync(partners);
        _partnerRepositoryMock.Setup(r => r.GetTotalSalesByPartnerIdAsync(1)).ReturnsAsync(50000m);
        _partnerRepositoryMock.Setup(r => r.GetTotalSalesByPartnerIdAsync(2)).ReturnsAsync(100000m);

        // Act
        var result = await _partnerService.GetAllPartnersAsync();

        // Assert
        Assert.NotNull(result);
        Assert.Equal(2, result.Count);
        Assert.Equal("Partner 1", result[0].Name);
        Assert.Equal("Partner 2", result[1].Name);
        Assert.Equal(50000m, result[0].TotalSalesAmount);
        Assert.Equal(100000m, result[1].TotalSalesAmount);
        Assert.Equal(10, result[1].DiscountPercent); // 10% for 100000
    }

    [Fact]
    public async Task GetPartnerByIdAsync_ReturnsNull_WhenPartnerNotFound()
    {
        // Arrange
        _partnerRepositoryMock.Setup(r => r.GetByIdAsync(It.IsAny<int>())).ReturnsAsync((Partner?)null);

        // Act
        var result = await _partnerService.GetPartnerByIdAsync(999);

        // Assert
        Assert.Null(result);
```

Это **тесты для сервиса управления партнёрами**. Они проверяют, что PartnerService правильно работает с базой данных (через моки).

Что тестируем

GetAllPartnersAsync_ReturnsViewModelsWithTotalSales

Проверяет, что сервис возвращает список всех партнёров, и у каждого правильно подставлена сумма продаж и рассчитана скидка.

Настройка мока: GetAllAsync() возвращает список партнёров, а GetTotalSalesByPartnerIdAsync(id) возвращает разную сумму для каждого ID (например, 50000 для первого, 100000 для второго).

GetPartnerByIdAsync_ReturnsNull_WhenPartnerNotFound

Проверяет, что если в базе нет партнёра с таким ID, сервис корректно вернёт null, а не упадёт с ошибкой.

Настройка мока: GetByIdAsync(It.IsAny<int>()) возвращает null для любого запроса.

GetPartnerByIdAsync_ReturnsViewModel_WhenPartnerExists

Проверяет, что при поиске существующего партнёра сервис возвращает заполненную модель с правильными данными и рассчитанной скидкой.

Настройка мока: GetByIdAsync(1) возвращает тестового партнёра, а GetTotalSalesByPartnerIdAsync(1) возвращает сумму продаж (например, 25000).

AddPartnerAsync_CallsRepository_AddsPartner

Проверяет, что при добавлении нового партнёра сервис вызывает метод сохранения в репозитории и возвращает результат с присвоенным ID.

Настройка мока: AddAsync(It.IsAny<Partner>()) возвращает сохранённый объект с новым ID.

DeletePartnerAsync_CallsRepository_Delete

Проверяет, что при удалении сервис действительно вызывает метод удаления в репозитории с правильным ID.

Настройка мока: не требуется, проверяется только факт вызова через Verify.

GetPartnerTypesAsync_ReturnsAllTypes

Проверяет, что сервис корректно возвращает список всех типов партнёров (например, «Розничный», «Оптовый») для заполнения выпадающего списка в интерфейсе.

Настройка мока: GetAllAsync() у репозитория типов возвращает подготовленный список.

GetPartnerSalesAsync_ReturnsSalesForPartner

Проверяет, что при запросе истории продаж сервис возвращает только те записи, которые принадлежат указанному партнёру.

Настройка мока: GetByPartnerIdAsync(1) возвращает список тестовых продаж.

Диаграмма классов

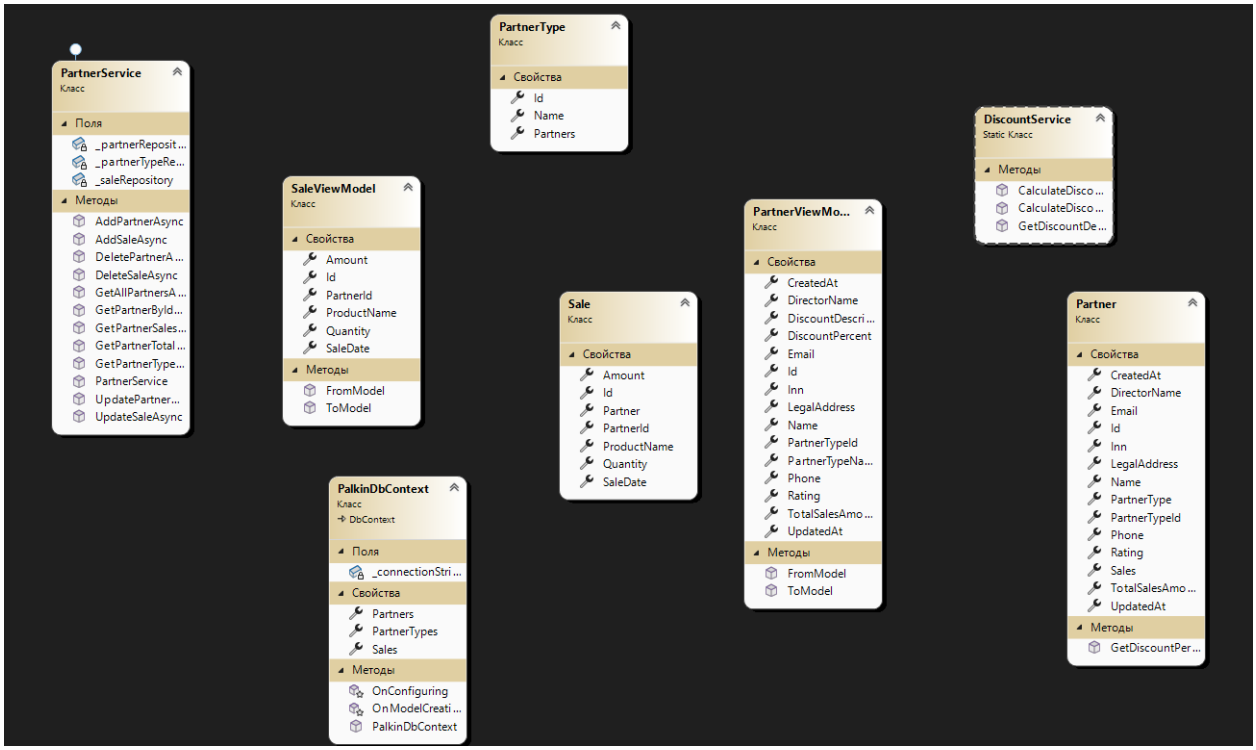
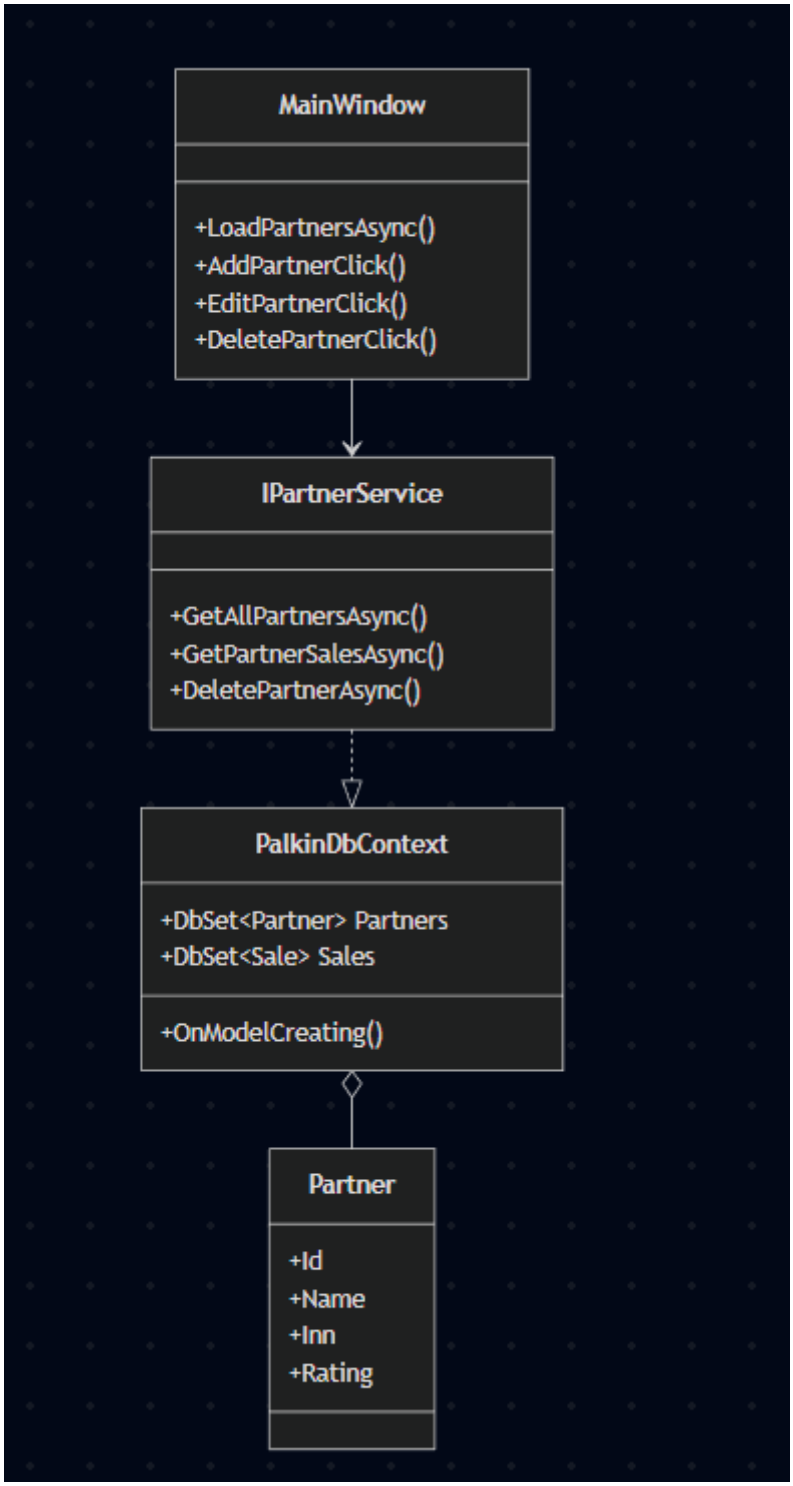
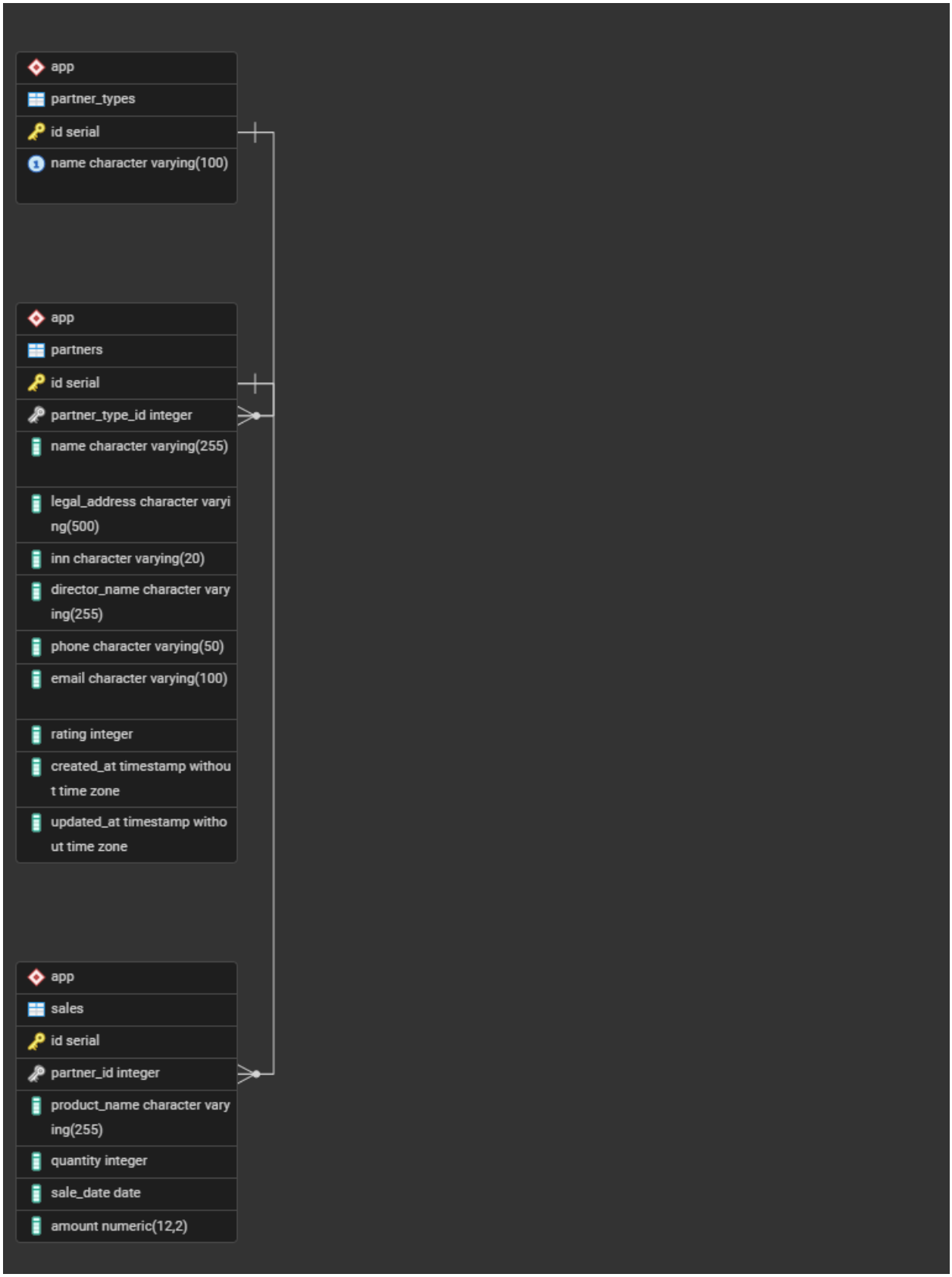


Схема базы данных



ER-диаграмма



Реализация ключевых модулей

Модуль доступа к данным (PalkinDbContext)

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.HasDefaultSchema("app");

    modelBuilder.Entity<Partner>(entity =>
    {
        entity.ToTable("partners");
        entity.Property(e => e.Name).IsRequired().HasMaxLength(255);
        entity.HasCheckConstraint("CK_Partners_Rating", "\"rating\" >= 0");
        entity.HasOne(e => e.PartnerType)
              .WithMany(e => e.Partners)
              .HasForeignKey(e => e.PartnerTypeId)
              .OnDelete(DeleteBehavior.Restrict);
    });
}
```

Особенности реализации:

- Использована схема app для изоляции таблиц
- Настроены Check Constraints для валидации на уровне БД
- Реализованы каскадные удаления для связанных записей (Sales)

Модуль бизнес-логики (PartnerService)

```
public async Task<List<PartnerViewModel>> GetAllPartnersAsync()
{
    var partners = await _partnerRepository.GetAllAsync();
    var result = new List<PartnerViewModel>();

    foreach (var partner in partners)
    {
        var totalSales = await _saleRepository.GetTotalSalesByPartnerIdAsync(partner.Id);
        result.Add(new PartnerViewModel
        {
            Id = partner.Id,
            Name = partner.Name,
            TotalSalesAmount = totalSales,
            DiscountPercent = DiscountService.CalculateDiscountPercent(totalSales)
        });
    }
    return result;
}
```

Метод сервиса для загрузки всех партнёров с расчётом их продаж и скидок.
Алгоритм работы:

1. Получает список партнёров из репозитория
2. Для каждого партнёра:
    - Запрашивает общую сумму продаж из SaleRepository
    - Создаёт PartnerViewModel с данными
    - Рассчитывает скидку через DiscountService
3. Возвращает список view-моделей

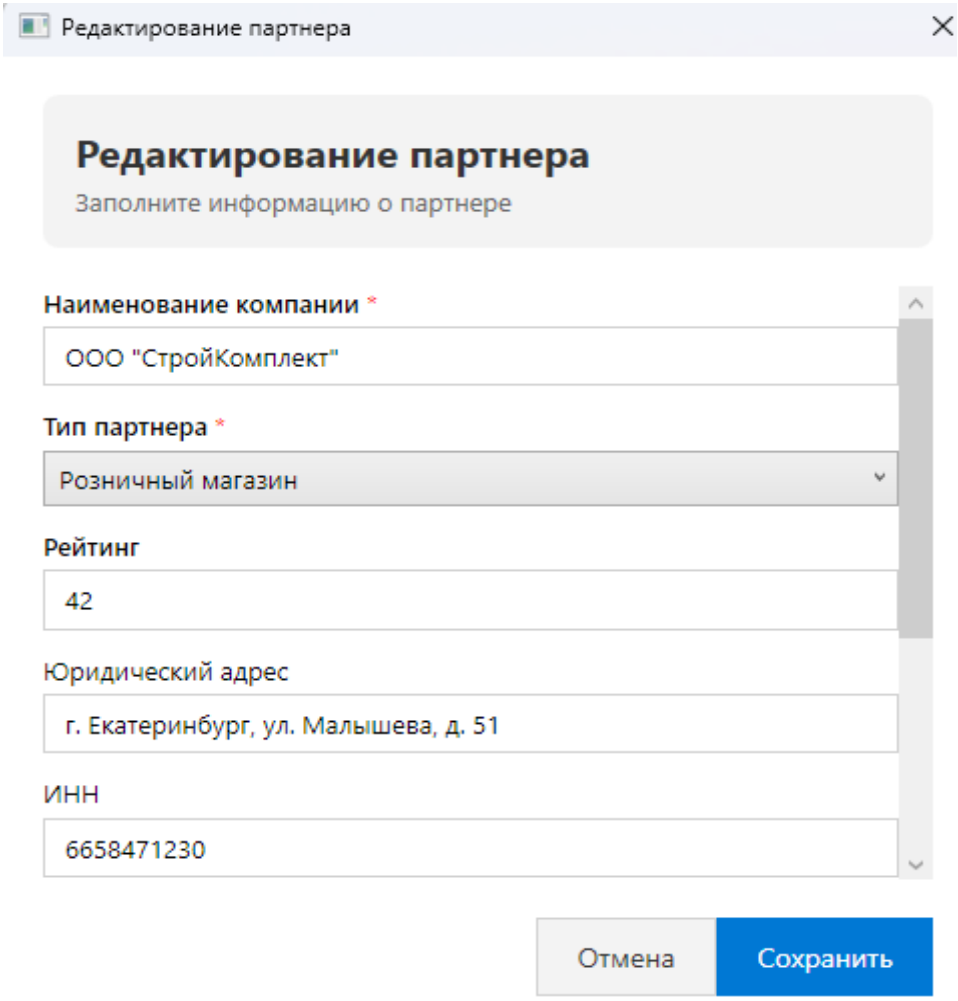
Ключевые особенности:

- Автоматический расчёт — скидка вычисляется на лету

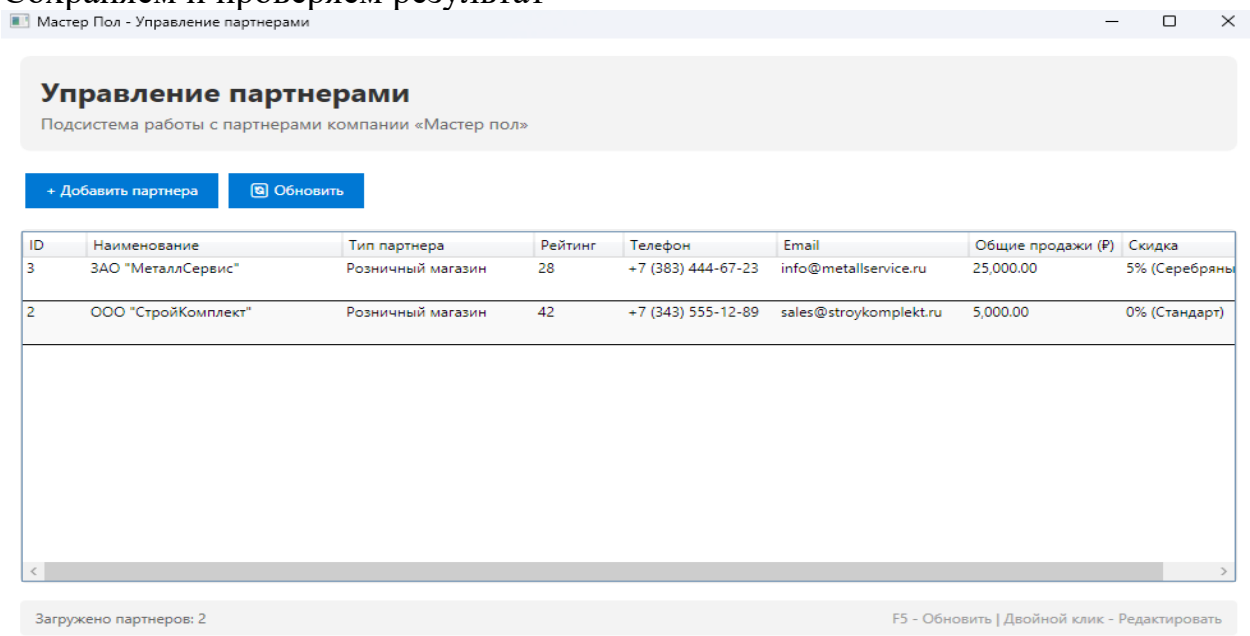
Тестирование программы

Добовления партнера

Сначала вводим данные партнера



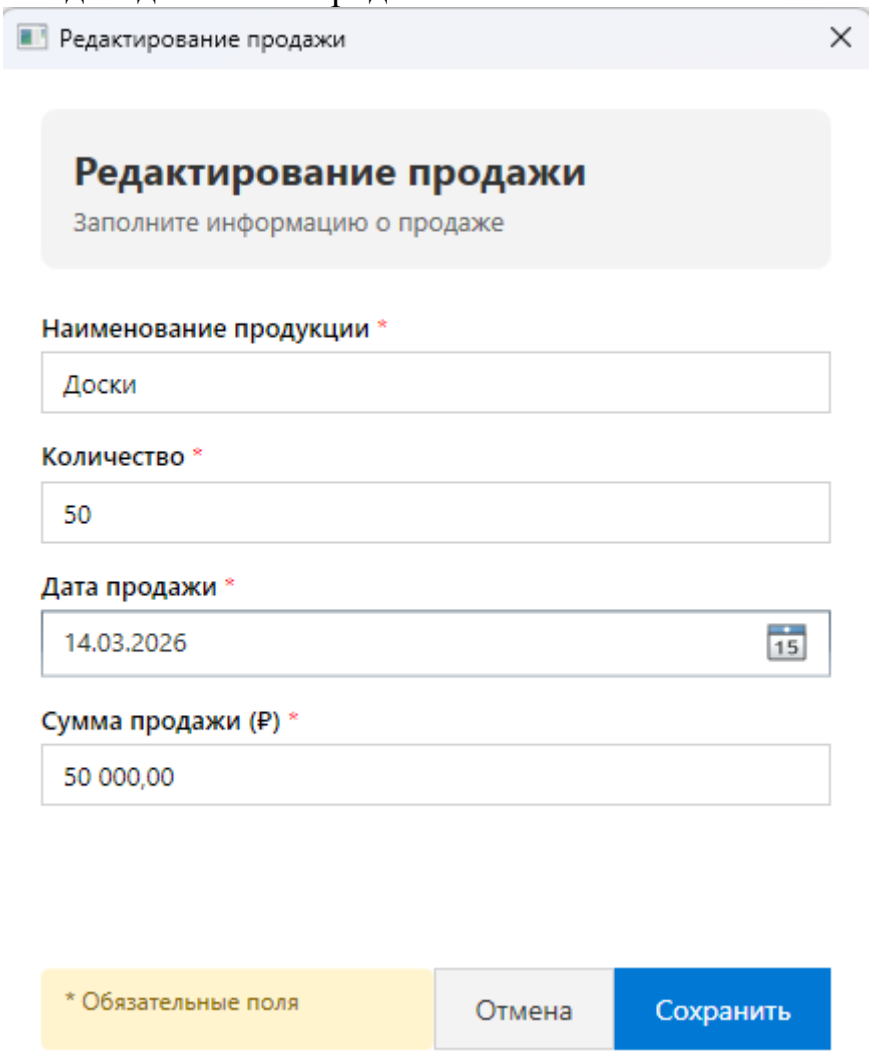
Сохраняем и проверяем результат



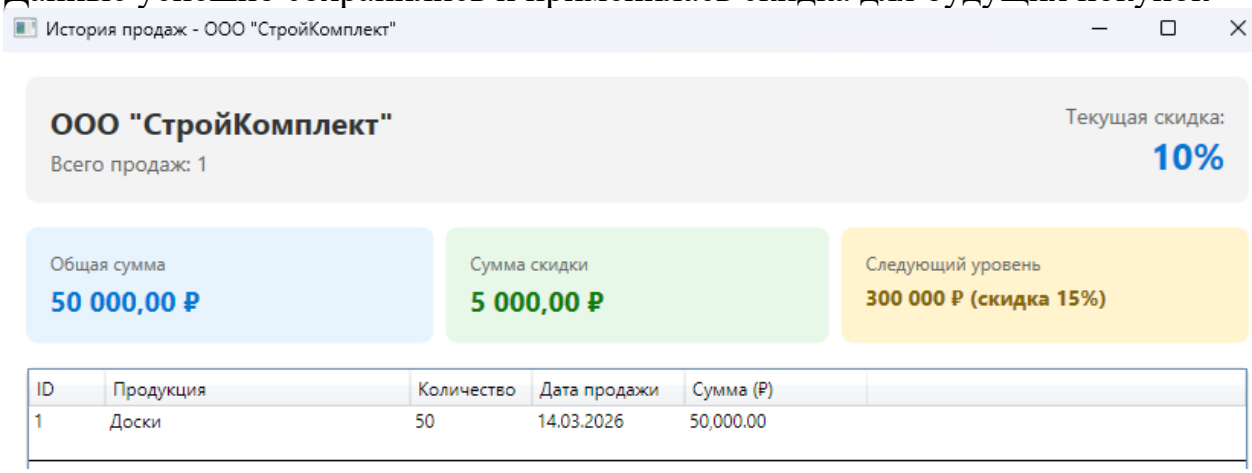
Партнер успешно сохранилась

Проверка продаж и скидки

Вводим данные по продаже



Данные успешно сохранились и применилась скидка для будущих покупок



Заключение

В ходе работы была создана автоматизированная подсистема управления партнёрами для компании «Мастер Пол» на платформе .NET 8. Система решает задачи автоматического расчёта скидок, учёта продаж и упрощения работы с контрагентами.

Основой проекта стала база данных PostgreSQL, разработанная с учётом целостности информации: настроены внешние ключи, проверки на корректность значений и каскадное удаление связанных записей. Взаимодействие с данными реализовано через Entity Framework Core, что позволило сосредоточиться на бизнес-логике без написания прямых SQL-запросов.

Ключевым элементом системы является сервис расчёта скидок, который автоматически присваивает партнёру один из четырёх статусов (от «Стандарт» до «Премиум») в зависимости от объёма продаж. Логика вынесена в отдельный статический класс для удобства тестирования и повторного использования.

Для работы пользователей разработано десктопное приложение на WPF с современным интерфейсом. Оно предоставляет удобный список партнёров с отображением текущих показателей, а также позволяет редактировать данные, управлять продажами и отслеживать историю сделок через интуитивно понятное меню.

Выполненные задачи

1. **Спроектирована база данных** (PostgreSQL) для хранения информации о партнёрах, типах партнёров и истории продаж с соблюдением целостности данных.
2. **Реализована бизнес-логика** расчёта скидок в зависимости от объёма продаж (4 уровня: 0%, 5%, 10%, 15%).
3. **Разработано десктопное приложение** (WPF) с удобным интерфейсом для просмотра, добавления, редактирования и удаления партнёров.
4. **Написаны модульные тесты** (xUnit + Moq), покрывающие ключевые сценарии работы сервиса.

Разработанная система позволяет:

- Автоматизировать расчёт скидок и исключить человеческие ошибки.
- Вести учёт продаж в разрезе каждого партнёра.
- Быстро получать информацию о статусе и прогрессе партнёра до следующего уровня.
- Масштабировать функционал за счёт модульной архитектуры.

Как итог в результате была создана работоспособная информационная система, готовая к внедрению. Код покрыт тестами, документирован и соответствует современным стандартам разработки на .NET.